Instrumenting Java Code

Mateusz Rzeszutek

21st October 2025

Who am I?

I'm an emeritus maintainer of the OpenTelemetry Instrumentation for Java

What is instrumentation?

According to Wikipedia:

> *In computer programming, instrumentation is the act of modifying software so that analysis can be performed on it.*

What is instrumentation?

Java instrumentation techniques:

- Manual instrumentation
- Auto instrumentation
  - Code generation
  - Javaagents
  - Compile-time instrumentation
- ...

Manual instrumentation

How to instrument the following?

```
public interface Supplier<T> {
  T get();
}
```

$\rightarrowtail IDE$

Manual instrumentation

Some libraries expose hooks for instrumentation:

- OkHttp has Interceptors and EventListeners
- HikariCP has MetricsTrackerFactory
- ...

Others expose interfaces that can be proxied.

JDBC?

```
interface PreparedStatement {
  // ...
  boolean execute();
  boolean execute(String);
  boolean execute(String, int);
  boolean execute(String, int[]);
  boolean execute(String, String[]);
  int[] executeBatch();
  long[] executeLargeBatch();
  long executeLargeUpdate();
  long executeLargeUpdate(String);
  long executeLargeUpdate(String, int);
  long executeLargeUpdate(String, int[]);
  long executeLargeUpdate(String, String[]);
  //
```

Code generation: java.util.Proxy

```
PreparedStatement ps = // ...;

var timedStatement = Proxy.newProxyInstance(
    ps.getClass().getClassLoader(),
    new Class[]{ PreparedStatement.class },
    new TimedStatementHandler(ps));
);
```

$\rightarrowtail IDE$

Code generation: `java.util.Proxy`

`java.util.Proxy` can generate implementations for interfaces, but not classes or abstract classes.

Code generation: AOP, Spring, AspectJ

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Timed {
}
```

$\rightarrowtail IDE$

Code generation: other utilities

- java.util.Proxy
- AspectJ
- CGLib
- Javassist
- ASM
- Class-File API
- Byte Buddy
- ...

Code generation

Task: generate a dynamic class that returns `"Hello World"` when `.toString()` is called.

Code generation: ASM

```
var cw = new ClassWriter(ClassWriter.COMPUTE_MAXS);
cw.visit(Opcodes.V24, Opcodes.ACC_PUBLIC,
    "org.example.Generated", null, null, null);
var mw = cw.visitMethod(Opcodes.ACC_PUBLIC,
    "toString", "()Ljava/lang/String;", null, null);
mw.visitCode();
mw.visitLdcInsn("Hello World");
mw.visitInsn(Opcodes.ARETURN);
mw.visitEnd();
cw.visitEnd();
var bytes = cw.toByteArray();
```

Code generation: Class-File

```
var bytes = ClassFile.of()
    .build(
        ClassDesc.of("org.example.Generated"),
        cb -> cb.withMethodBody(
            "toString",
            MethodTypeDesc.of(
                ClassDesc.of("java.lang.String")),
            ClassFile.ACC_PUBLIC,
            code -> code.ldc("Hello World").areturn()
        ));
```

JVM specification

For the interested, look up the JVM specification chapter 6: The Java Virtual Machine Instruction Set

Code generation: Byte Buddy

```
try (var unloadedClass = new ByteBuddy()
    .subclass(Object.class)
    .name("org.example.Generated")
    .method(ElementMatchers.named("toString"))
    .intercept(FixedValue.value("Hello World"))
    .make()) {

  var bytes = unloadedClass.getBytes();
}
```

$\rightarrowtail IDE$

Javaagent

A javaagent is a special type of Java application that can be attached to a JVM process and instrument it via the Java Instrumentation API.

```
java -javaagent:opentelemetry-javaagent.jar
     -jar my-app.jar
```

Javaagent

Javaagent main class:

```
class MyJavagent {
  public static void premain(
        String agentArgs,
        Instrumentation inst) {
    System.out.println("before main()");
  }
}
```

JAR manifest attributes:

```
Premain-Class: org.example.MyJavaagent
```

Loading, Linking, Initializing

1. Loading

- Classloaders: bootstrap, platform, system – and others;
- Super classes and interfaces are resolved at this point too;
- This is where javaagent instrumentation kicks in;
- `Class.forName(name, false, cl)` will load, but not link & initialize the class.

Loading, Linking, Initializing

2. Linking

- Verification: class is valid and does not violate the semantics of Java;
- Preparation: allocates and prepares static fields (does not execute `<clinit>`!);
- Resolution: bytecode instructions that refer to other symbols in the constant pool (e.g. fields, methods) must be resolved.

Loading, Linking, Initializing

3. Initializing

- Executes the `<clinit>` method (static class initialization);
- Initialization is synchronized, which is often used in e.g. the `Holder` singleton pattern;
- Triggered by some of the JVM instructions (`new`, `putstatic`, `getstatic`, `invokestatic`), method handles, reflection, and subclass initialization.

JVM specification

For the interested, look up the JVM specification chapter 5: Loading, Linking, and Initializing

Java Instrumentation API

```
interface Instrumentation {
  void addTransformer(ClassFileTransformer, boolean);
  boolean removeTransformer(ClassFileTransformer);
  void retransformClasses(Class<?>...);
  void redefineClasses(ClassDefinition...);
  // ...
}
```

Java Instrumentation API

```
interface ClassFileTransformer {
  byte[] transform(
      Module module,
      ClassLoader loader,
      String className,
      Class<?> classBeingRedefined,
      ProtectionDomain protectionDomain,
      byte[] classfileBuffer);
   // ...
}
```

$\rightarrowtail IDE$

Compile-time instrumentation

What if you can't use javaangents?

Need to instrument Android applications?

Or need to modify the bytecode and recompile a library?

Compile-time instrumentation

```
plugins {
  id("net.bytebuddy.byte-buddy-gradle-plugin")
}

byteBuddy {
  transformation {
    plugin = MyCompileTimePlugin::class.java
  }
}
```

$\rightarrowtail IDE$

Compile-time instrumentation

We can check the compiled code for changes!

```
javap -c App
```

$\rightarrowtail sh$

Thank you!

Presentation sources

https://github.com/mateuszrzeszutek/instrumenting-java-code

QR Code

Q&A